

2

Consumir modelos por API

Cómo consumir un modelo de IA por API REST desde Python, con autenticación segura, manejo de errores y un cliente reutilizable, documentado y probado.

APRENDIZAJE ESPERADO 2 · TEXTUAL DEL PLAN FORMATIVO

UTILIZAR UN MODELO DE IA A TRAVÉS DE API REST UTILIZANDO SOLICITUDES HTTP Y AUTENTICACIÓN, BASÁNDOSE EN LA DOCUMENTACIÓN OFICIAL DE OPENAI Y HUGGING FACE.

PLAN FORMATIVO

PF1822 · Especialización en Desarrollo con IA

MÓDULO

MA04576 · INTRODUCCIÓN A MODELOS GENERATIVOS Y CONSUMO POR API · 21 h

CASO DE ESTUDIO

Nube Sur, empresa ficticia

TIEMPO DE LECTURA

20 minutos, más la autocomprobación

Qué vas a lograr

Al terminar esta lectura tendrás la base para demostrar los criterios de evaluación del aprendizaje esperado 2, textuales del plan formativo:

- 2.1** EXPLICA LA ESTRUCTURA Y PRINCIPALES SECCIONES DE LOS ENDPOINTS DE GENERACIÓN Y EMBEDDINGS DE ACUERDO CON LA DOCUMENTACIÓN OFICIAL DE OPENAI Y HUGGING FACE.

- 2.2** IMPLEMENTA LLAMADAS HTTP GET Y POST A ENDPOINTS DE COMPLETIONS Y EMBEDDINGS PARA EL CONSUMO DE SERVICIOS, DE ACUERDO CON LA DOCUMENTACIÓN OFICIAL.

- 2.3** IMPLEMENTA UNA FUNCIÓN O CLASE MODULAR EN PYTHON QUE ENCAPSULA LA LLAMADA AL API Y PERMITE REUTILIZARLA CON DISTINTOS PARÁMETROS, DOCUMENTADA CON DOCSTRINGS Y PRUEBAS UNITARIAS SIMPLES.

Cómo está organizada

- | | |
|--|---|
| <ul style="list-style-type: none"> 1 REST en dos métodos <hr/> 2 Leer la documentación oficial <hr/> 3 Endpoints de generación y de embeddings <hr/> 4 Autenticación y variables de entorno <hr/> 5 requests y httpx <hr/> | <ul style="list-style-type: none"> 6 Cuerpos JSON y lectura de la respuesta <hr/> 7 Errores del servicio <hr/> 8 Modularizar y documentar <hr/> 9 Pruebas unitarias sin conexión <hr/> + En síntesis, práctica, autocomprobación y glosario <hr/> |
|--|---|

Contenidos del plan y dónde se tratan

Contenido del plan formativo (textual)	Dónde
2. INTEGRACIÓN Y CONSUMO DE APIS	Antes de empezar
PRINCIPIOS REST Y MÉTODOS HTTP: GET Y POST.	Sección 1
DOCUMENTACIÓN OFICIAL: ENLACES Y ORGANIZACIÓN DE LOS MANUALES DE OPENAI Y HUGGING FACE INFERENCE API.	Sección 2
ENDPOINTS DE GENERACIÓN Y EMBEDDINGS.	Sección 3
GESTIÓN DE AUTENTICACIÓN Y VARIABLES DE ENTORNO.	Sección 4
LIBRERÍAS: REQUESTS Y HTTPX.	Sección 5
EJEMPLOS DE USO: PARÁMETROS, CUERPOS JSON Y PARSING DE RESPUESTA.	Sección 6
MANEJO BÁSICO DE ERRORES Y RESPUESTAS DEL SERVICIO.	Sección 7
MODULARIZACIÓN DE FUNCIONES O CLASES PARA CONSUMO DE APIS.	Sección 8
DOCUMENTACIÓN CON DOCSTRINGS.	Sección 8
PRUEBAS UNITARIAS SIMPLES PARA FUNCIONES DE CONSUMO DE APIS.	Sección 9

Antes de empezar

CONTENIDO DEL PLAN 2. INTEGRACIÓN Y CONSUMO DE APIS

Hablar con un modelo de IA desde una aplicación es, en el fondo, hacer una **solicitud HTTP bien hecha**: la dirección correcta, el método correcto, una clave válida y un cuerpo con el formato que el servicio espera. Lo que distingue a un script de prueba de un código profesional es todo lo demás: dónde se guarda la clave, qué pasa cuando el servicio falla, cómo se reutiliza el código y cómo se prueba sin gastar.

En esta lectura vas a construir, paso a paso, las piezas que forman `ClienteIA`, la clase que usa el resumidor de tickets de **Nube Sur**, la empresa ficticia del módulo. Al final vas a poder leer la documentación oficial de un endpoint, llamarlo desde Python con GET o POST, interpretar su respuesta y encapsular todo en un módulo con docstrings y pruebas unitarias.

UNA LLAMADA A UN MODELO, DE PRINCIPIO A FIN



Los ejemplos usan la API de OpenAI porque su formato lo replican muchos proveedores; la lógica es la misma con Hugging Face. Puedes seguirlos en el notebook guiado del curso.

1 REST en dos métodos

CONTENIDO DEL PLAN PRINCIPIOS REST Y MÉTODOS HTTP: GET Y POST.

REST es un estilo para diseñar APIs sobre HTTP. Sus principios, en lo básico, son tres.

Cada **recurso** tiene una dirección (URL), como `/v1/models`. Cada solicitud es

independiente: lleva todo lo necesario, incluida la autenticación, y el servidor no recuerda las anteriores. Y la respuesta trae un **código de estado** que dice cómo salió, además de un cuerpo, casi siempre en JSON.

De los métodos HTTP, dos cubren casi todo el consumo de modelos. **GET** sirve para **leer**: pide un recurso sin enviar datos en el cuerpo, como la lista de modelos disponibles. **POST** sirve para **enviar datos y pedir un resultado**: el cuerpo lleva el texto y los parámetros, y la respuesta trae lo generado. Los códigos de estado se leen por su primera cifra: 2xx es éxito, 4xx es un error de quien hace la solicitud y 5xx es un error del servicio.

Método	Para qué	Ejemplo en la API de OpenAI
GET	Leer un recurso	GET <code>/v1/models</code> : lista los modelos disponibles
POST	Enviar datos y obtener un resultado	POST <code>/v1/chat/completions</code> : genera una respuesta

Método	Para qué	Ejemplo en la API de OpenAI
POST	Enviar datos y obtener un resultado	POST /v1/embeddings: obtiene vectores de textos

EJEMPLO · NUBE SUR · Dos solicitudes del resumidor

Al arrancar, el servidor de Nube Sur hace un **GET** a /v1/models para confirmar que el modelo configurado existe y que la clave funciona. Después, por cada ticket, hace un **POST** a /v1/chat/completions con el texto limpio y recibe el resumen. La primera lee; la segunda envía y produce.

2 Leer la documentación oficial

CONTENIDO DEL PLAN DOCUMENTACIÓN OFICIAL: ENLACES Y ORGANIZACIÓN DE LOS MANUALES DE OPENAI Y HUGGING FACE INFERENCE API.

La documentación oficial es la fuente de verdad de una API. Los tutoriales envejecen; la referencia oficial se actualiza. Por eso conviene saber cómo está organizada y qué buscar en ella.

La documentación de **OpenAI** (platform.openai.com/docs) tiene tres partes que usarás siempre. Las **guías** explican cada capacidad con ejemplos: generación de texto, embeddings, buenas prácticas. La **referencia de la API** describe cada endpoint con su URL, método, parámetros, cuerpo de la solicitud, objeto de respuesta y ejemplos en varios lenguajes. Y las páginas de **modelos** y de **límites** dicen qué modelos existen y cuántas solicitudes admite cada cuenta.

La documentación de **Hugging Face** (huggingface.co/docs) se organiza distinto, porque su oferta es otra. Cada modelo del Hub tiene una **ficha** (*model card*) que explica para qué sirve, cómo usarlo y con qué licencia. La API de inferencia, hoy documentada como *Inference Providers*, se organiza **por tarea**: generación de texto, conversación, resumen o extracción de características (embeddings). Ofrece además un endpoint compatible con el formato de chat de OpenAI, lo que facilita cambiar de proveedor.

	API de OpenAI	Hugging Face
Qué ofrece	Modelos propios de la empresa por API	Miles de modelos abiertos y servicios para ejecutarlos por API o en la propia máquina
Autenticación	Clave de API en el encabezado	Token de acceso en el encabezado
Cuándo conviene	Calidad alta con poca configuración	Elegir o cambiar de modelo, usar modelos abiertos, ejecutar localmente

EJEMPLO · NUBE SUR · Qué buscar en una entrada de la referencia

Antes de programar la llamada de embeddings, el equipo abre su entrada en la referencia y anota cinco cosas: la **URL y el método** (POST /v1/embeddings), la **autenticación** (clave en el encabezado), los **parámetros obligatorios** del cuerpo (model e input), la **forma de la respuesta** (una lista data con un embedding por texto) y los **errores** posibles. Con esas cinco respuestas se puede escribir el código.

ERROR FRECUENTE · Copiar un ejemplo antiguo

Los nombres de modelos, las URL y algunos parámetros cambian con frecuencia. Un ejemplo de hace un año puede usar un modelo retirado o un parámetro que ya no se acepta. Verifica siempre contra la referencia vigente antes de escribir o publicar código.

3 Endpoints de generación y de embeddings

CONTENIDO DEL PLAN ENDPOINTS DE GENERACIÓN Y EMBEDDINGS.

Un **endpoint** es una URL de la API que ofrece una operación. Para este módulo importan dos. El de **generación** de chat, POST /v1/chat/completions, recibe una lista de mensajes con roles y devuelve la respuesta del modelo. OpenAI ofrece también una API más reciente, *Responses*, con otro formato; aquí se usa chat completions porque muchos proveedores la replican.

```
{
  "model": "<modelo de chat>",
  "messages": [
    {"role": "system", "content": "Resume el ticket en una oración."},
    {"role": "user",
      "content": "Ticket: No puedo descargar la factura de agosto."}
  ],
  "temperature": 0.2,
  "max_tokens": 60
}
```

La respuesta trae el texto en `choices[0].message.content`, el motivo de término en `finish_reason` y el consumo en `usage`, con los tokens de entrada (`prompt_tokens`), de salida (`completion_tokens`) y el total.

El endpoint de **embeddings**, POST /v1/embeddings, recibe en `input` un texto o una lista de textos y devuelve en `data` un **vector** de números por cada uno, con su posición en `index`. Esos vectores permiten comparar textos por su significado: dos tickets sobre el mismo problema tienen vectores cercanos.

EJEMPLO · NUBE SUR · Tickets parecidos

Nube Sur envía al endpoint de embeddings tres frases: "No puedo iniciar sesión", "El sistema no me deja entrar" y "El PDF sale con tildes rotas". Las dos primeras reciben vectores muy cercanos; la tercera, uno lejano. Así el equipo agrupa tickets repetidos aunque estén escritos con palabras distintas.

4 Autenticación y variables de entorno

CONTENIDO DEL PLAN GESTIÓN DE AUTENTICACIÓN Y VARIABLES DE ENTORNO.

Las APIs de modelos se autentican con una **clave** que identifica tu cuenta y a la que se cargan los consumos. Se envía en cada solicitud, en el encabezado `Authorization`, con la palabra `Bearer` antes de la clave. En Hugging Face el mecanismo es el mismo, con un token de acceso.

La clave **nunca** se escribe en el código, en un notebook, en el repositorio ni en una captura de pantalla. Vive en una **variable de entorno**, que es un valor que el sistema operativo entrega al programa al ejecutarlo. En tu computador se define en la terminal o en un archivo `.env` que está listado en `.gitignore`; en Colab, en los *Secrets* del notebook; en un servidor, en su configuración. El código solo lee el nombre.

```
import os

clave_api = os.environ.get("OPENAI_API_KEY")
if not clave_api:
    raise ValueError("Falta OPENAI_API_KEY: definela en el entorno, "
                    "nunca en el código.")
encabezados = {"Authorization": f"Bearer {clave_api}"}
```

EJEMPLO · NUBE SUR · La misma línea en tres lugares

En el notebook de Colab, la clave se lee de los *Secrets* con `userdata.get("OPENAI_API_KEY")`. En el computador de quien desarrolla, de una variable de entorno. En el servidor de Nube Sur, de su configuración de despliegue. El código de `ClienteIA` es el mismo en los tres: solo cambia dónde se definió el valor.

ERROR FRECUENTE · La clave en el repositorio

Una clave escrita en un archivo y subida a un repositorio queda expuesta aunque se borre después, porque sigue en el historial. Si pasa, la única corrección es **revocarla** en el panel del proveedor y crear una nueva. Por eso la regla es preventiva: la clave nunca toca un archivo versionado.

5 requests y httpx

CONTENIDO DEL PLAN LIBRERÍAS: REQUESTS Y HTTPX.

En Python, dos bibliotecas cubren el consumo de APIs. **requests** es la más conocida: simple, estable y con abundante documentación. **httpx** tiene una interfaz casi igual y suma tres ventajas útiles para este módulo: **tiempos de espera** activos por defecto, soporte para programación asíncrona y una forma sencilla de **simular la API** en las pruebas, con `httpx.MockTransport`.

	requests	httpx
Solicitud POST con JSON	<code>requests.post(url, headers=h, json=cuerpo, timeout=30)</code>	<code>httpx.post(url, headers=h, json=cuerpo, timeout=30)</code>
Tiempo de espera por defecto	Ninguno: hay que indicarlo	5 segundos
Cliente reutilizable	<code>requests.Session()</code>	<code>httpx.Client(base_url=..., timeout=...)</code>
Asincronía	No	Sí, con <code>httpx.AsyncClient</code>
Simular la API en pruebas	Con bibliotecas adicionales	Incluido: <code>httpx.MockTransport</code>

Un **cliente reutilizable** guarda la URL base, los encabezados y el tiempo de espera, y mantiene abierta la conexión entre solicitudes. Es más ordenado y más rápido que repetir los datos en cada llamada.

EJEMPLO · NUBE SUR · El cliente de Nube Sur

`httpx.Client(base_url="https://api.openai.com/v1", timeout=30.0)` crea el cliente una sola vez. Después, `cliente.get("/models")` y `cliente.post("/chat/completions", json=cuerpo)` usan la misma conexión, la misma URL base y el mismo encabezado de autenticación.

ERROR FRECUENTE · Solicitudes sin tiempo de espera

Con `requests`, una solicitud sin `timeout` puede quedar esperando indefinidamente si el servicio no responde, y con ella el programa completo. Indica siempre un tiempo de espera, y decide qué hacer cuando se cumple.

6 Cuerpos JSON y lectura de la respuesta

CONTENIDO DEL PLAN EJEMPLOS DE USO: PARÁMETROS, CUERPOS JSON Y PARSING DE RESPUESTA.

Consumir un endpoint tiene siempre los mismos pasos. Primero, **armar el cuerpo** como un diccionario de Python con los parámetros que pide la documentación. Segundo, **enviarlo con `json=`**, que lo convierte a JSON y agrega el encabezado de tipo de contenido. Tercero,

revisar el código de estado. Y cuarto, leer la respuesta con `respuesta.json()` y navegar hasta los campos que necesitas.

```
cuerpo = {
    "model": os.environ["MODELO_CHAT"],
    "messages": [
        {"role": "system",
         "content": "Resume el ticket en una oración de máximo 25 palabras."},
        {"role": "user", "content": "Ticket: " + ticket_limpio},
    ],
    "temperature": 0.2,
    "max_tokens": 60,
}
respuesta = cliente.post("/chat/completions", json=cuerpo)
if respuesta.status_code != 200:
    raise RuntimeError(
        f"HTTP {respuesta.status_code}: {respuesta.text[:300]}")
datos = respuesta.json()
resumen = datos["choices"][0]["message"]["content"]
tokens = datos["usage"]["total_tokens"]
```

EJEMPLO · NUBE SUR · Leer lo que importa

Para el ticket T3, sobre la API de pagos con error 504, el servidor lee dos cosas: el resumen, en `choices[0].message.content`, y el total de tokens, en `usage.total_tokens`. El resumen se muestra en la mesa de ayuda; los tokens se suman para controlar el costo diario del resumidor.

ERROR FRECUENTE · Leer choices de una respuesta de error

Si la clave es inválida, la respuesta no trae `choices`, sino un objeto `error`. Leer `datos["choices"]` sin revisar antes el código de estado produce un `KeyError` que oculta la causa real. Siempre primero `status_code`, después el contenido.

7 Errores del servicio

CONTENIDO DEL PLAN MANEJO BÁSICO DE ERRORES Y RESPUESTAS DEL SERVICIO.

Un servicio externo puede fallar, y el código tiene que decidir qué hacer en cada caso. El código de estado dice de quién es el problema y si tiene sentido reintentar:

Código	Qué significa	Qué hacer
400	Solicitud mal formada: falta un parámetro o tiene un valor inválido	Corregir el cuerpo; reintentar no sirve
401	Clave ausente o inválida	Revisar la variable de entorno; reintentar no sirve
404	El recurso o el modelo no existe	Revisar la URL y el nombre del modelo

Código	Qué significa	Qué hacer
429	Demasiadas solicitudes o cuota agotada	Esperar y reintentar, con esperas crecientes
5xx	Falla del proveedor	Reintentar unas pocas veces; si persiste, avisar

Para los errores que sí vale la pena reintentar, como 429 o 5xx, la práctica es esperar cada vez más: 1, 2 y 4 segundos, por ejemplo, con un máximo de intentos. Si la respuesta trae el encabezado `Retry-After`, indica cuánto esperar. A esto se suman los errores sin código de estado: un tiempo de espera cumplido o una conexión que no se pudo establecer.

EJEMPLO · NUBE SUR · Un error que se entiende

Con una clave vencida, `ClienteIA` no se cae con un `KeyError`: lanza `ErrorAPI` con el mensaje "HTTP 401" seguido del texto del servicio. Quien lo lee en el registro sabe en segundos que el problema es la clave y no el código. Un mensaje claro vale más que un programa que se detiene sin explicación.

8 Modularizar y documentar

CONTENIDO DEL PLAN MODULARIZACIÓN DE FUNCIONES O CLASES PARA CONSUMO DE APIS. · DOCUMENTACIÓN CON DOCSTRINGS.

Modularizar es reunir en una sola pieza de código todo lo que tiene que ver con la API: la URL base, la autenticación, el tiempo de espera, la revisión de errores y cada operación. El resto de la aplicación llama a métodos con nombres claros, como `generar()` o `embeddings()`, y no sabe nada de HTTP. Si mañana cambia el proveedor o el formato, se cambia un solo archivo.

La clase `ClienteIA` de Nube Sur tiene un método por operación y uno interno, `_revisar`, que todos usan para convertir los errores HTTP en `ErrorAPI`. Además, recibe opcionalmente un cliente `http` ya construido: así, las pruebas pueden entregarle una API simulada.

Las **docstrings** son el texto entre comillas triples al comienzo de una clase o función. Explican qué hace, qué recibe (*Args*), qué devuelve (*Returns*) y qué errores puede lanzar. Python las guarda con el código: `help(ClienteIA.generar)` las muestra y los editores las presentan al escribir.

```
def generar(self, mensajes: list[dict], temperature: float = 0.2,
            max_tokens: int = 300) -> dict:
    """Envía una conversación al endpoint de chat (POST /chat/completions).

    Args:
        mensajes: lista de {"role": "system" | "user" | "assistant",
                           "content": str}.
        temperature: aleatoriedad de la respuesta; baja para resúmenes
                     consistentes.
        max_tokens: máximo de tokens de la respuesta.

    Returns:
        {"texto": str, "tokens": dict} con el texto generado y el uso
        de tokens.
    """
```

EJEMPLO · NUBE SUR · Reutilizar con distintos parámetros

El mismo método `generar()` sirve para el resumen, con `temperature` 0,2 y 60 tokens, y para el borrador de respuesta al cliente, con `temperature` 0,7 y 300 tokens. La aplicación cambia los parámetros; la lógica de la llamada, la autenticación y los errores no se repite.

9 Pruebas unitarias sin conexión

CONTENIDO DEL PLAN PRUEBAS UNITARIAS SIMPLES PARA FUNCIONES DE CONSUMO DE APIS.

Una **prueba unitaria** verifica una pieza de código de forma aislada. Probar un cliente de API contra el servicio real tiene tres problemas: cuesta dinero, depende de internet y sus respuestas varían. La solución es **simular la API**: con `httpx.MockTransport`, el cliente recibe respuestas falsas que tú defines, sin salir del computador.

```
def test_error_http_se_informa_con_su_codigo():
    cliente = cliente_simulado({"error": {"message": "clave inválida"}},
                               estado=401)
    with pytest.raises(ErrorAPI) as error:
        cliente.generar([{"role": "user", "content": "hola"}])
    assert error.value.estado == 401
```

Las pruebas de un cliente de API suelen verificar cuatro cosas: que la solicitud va al endpoint correcto con el método correcto, que lleva el encabezado de autenticación, que la respuesta se lee bien y que los errores se informan con su código. Se ejecutan con `pytest` en segundos, cada vez que cambias el código.

EJEMPLO · NUBE SUR · Las pruebas de ClienteIA

El archivo `test_cliente_ia.py` de Nube Sur comprueba que `generar()` hace un POST a `/v1/chat/completions` con `Bearer` y devuelve texto y tokens; que `embeddings()` respeta el orden de entrada aunque la API los devuelva desordenados; que `listar_modelos()` usa GET; que un 401 lanza `ErrorAPI`, y que sin clave la clase no arranca. Son cinco pruebas y ninguna usa internet.

ERROR FRECUENTE · Pruebas que llaman a la API real

Una prueba que llama al servicio real falla sin internet, gasta en cada ejecución y puede dar resultados distintos cada vez. Además obliga a tener la clave en el ambiente de pruebas. Simula la API en las pruebas unitarias y deja las llamadas reales para una prueba manual controlada.

CIERRE

En síntesis

- En REST, cada recurso tiene una URL; GET lee y POST envía datos para obtener un resultado; el código de estado dice cómo salió.
- La referencia oficial es la fuente de verdad: en cada endpoint busca URL, método, autenticación, cuerpo, respuesta y errores.
- Chat completions recibe mensajes con roles y devuelve `choices` y `usage`; embeddings devuelve un vector por texto en `data`.
- La clave va en el encabezado `Authorization: Bearer` y se lee de una variable de entorno, nunca del código.
- `httpx` y `requests` hacen lo mismo en lo básico; `httpx` suma tiempos de espera por defecto y simulación para pruebas.
- Siempre: revisar primero el código de estado y después leer la respuesta.
- Reintenta con esperas crecientes solo lo que puede resolverse solo, como 429 y 5xx.
- Encapsula la API en una clase documentada con docstrings y pruébala sin conexión.

SIGUIENTE PASO

Para practicar

- **Notebook guiado "Laboratorio de prompts", sección 0.** Tu primera llamada al modelo desde Colab, con la clave en *Secrets* y una comprobación de que no quedó escrita.
- **Actividad 1, "Un cliente de API para el resumidor", parte B.** Programas `ClienteIA` y al menos tres pruebas unitarias que corren sin conexión.
- **Video interactivo "Del prompt a la respuesta".** Lees una solicitud y una respuesta reales de la documentación.
- **Documentación oficial.** La referencia de OpenAI (platform.openai.com/docs) y la de Hugging Face (huggingface.co/docs).

Autocomprobación

Responde con tus palabras antes de mirar las respuestas. Si dudas en alguna, vuelve a la sección que se indica.

- 1 ¿Qué método HTTP usas para listar los modelos disponibles y cuál para obtener embeddings? ¿Por qué?

- 2 Tu programa falla con `KeyError: 'choices'` al llamar al endpoint de chat. ¿Qué pasó probablemente y cómo lo previenes?

- 3 ¿Por qué las pruebas unitarias de `ClienteIA` simulan la API en vez de llamar al servicio real?

RESPUESTAS

1. GET para listar modelos, porque solo lee un recurso. POST para embeddings, porque envía datos en el cuerpo (los textos) y pide un resultado (secciones 1 y 3).
2. El servicio devolvió un error, por ejemplo un 401 por una clave inválida, y la respuesta traía un objeto `error` en vez de `choices`. Se previene revisando `status_code` antes de leer el contenido y lanzando un error claro con el código (secciones 6 y 7).
3. Porque así corren sin internet, no gastan dinero, dan siempre el mismo resultado y no necesitan la clave. `httpx.MockTransport` entrega respuestas falsas definidas en la prueba (sección 9).

Glosario

API REST	Interfaz que expone recursos en URL y se usa con métodos HTTP como GET y POST, con respuestas casi siempre en JSON.
Bearer	Esquema de autenticación en que la clave se envía en el encabezado <code>Authorization</code> precedida de esa palabra.
Código de estado	Número de tres cifras de una respuesta HTTP que indica el resultado: 2xx éxito, 4xx error de la solicitud, 5xx error del servicio.
Docstring	Texto de documentación al comienzo de una clase o función de Python, que explica qué hace, qué recibe y qué devuelve.
Endpoint	URL de una API que ofrece una operación concreta, como <code>/v1/embeddings</code> .
httpx	Biblioteca de Python para hacer solicitudes HTTP, con tiempos de espera por defecto y simulación para pruebas.
Prueba unitaria	Prueba automática que verifica una pieza de código de forma aislada, como un método de un cliente de API.
Variable de entorno	Valor que el sistema entrega al programa al ejecutarlo; se usa para guardar claves fuera del código.